Reflection on the Two Advanced Topics

Step Sequencer project: Java Sound and Multithreading

Topic 1: Java Sound

Why we chose it and how it fits the project

A step sequencer only becomes an instrument when the grid actually produces sound, so audio playback is central to our project. The user builds drum patterns by activating steps for kick, snare, hi-hat and clap, and the application has to play the right sample at the moment each active step is reached. Without a reliable way to load and trigger audio, the interface would only be a visual pattern editor.

What we learned

We researched the `javax.sound.sampled` package. We learned how `AudioSystem` turns a WAV resource into an `AudioInputStream` and how that stream can be converted to a fixed `AudioFormat`. We learned that a `Clip` loads a whole sample into memory before playback, which gives low start-up latency and suits short drum hits, whereas a streaming line (`SourceDataLine`) suits long audio but must be fed continuously. We also learned that a Clip is stateful: it has a single playback position and holds a system audio resource that must be closed. Playing a sample therefore involves much more than choosing a file and calling a play method.

How we applied it

The samples are organised in one folder per instrument with three choices each, loaded from the classpath so the project runs from the submitted files without absolute paths. Every instrument has a dropdown to select its sample. All samples are converted to 44.1 kHz, 16-bit stereo PCM, so every sound plays in the same format regardless of how the original file was exported. For each instrument we keep four Clips loaded with the selected sample and use them in rotation: when a step fires, the next Clip is stopped, rewound with `setFramePosition(0)` and started. When the user selects another sample, the new Clips are loaded first and then swapped in under a lock, and the old ones are closed afterwards, so the playback thread never uses a Clip that is being replaced.

Decisions

We chose Clips over a streaming line because drum hits are short and need to start quickly, and because preloading keeps file reading out of the timing-critical loop. We chose four voices per instrument as a compromise: it lets a sound overlap with itself several times, while more voices would only use more memory and open audio lines for every sample. We also decided that changing a sample must be safe at any moment. Disabling the dropdowns during playback hides the problem in the interface but does not make it impossible in code, so the Clip handling itself is protected.

Challenges and what we learned

Our first version used one Clip per instrument. At fast tempos a hi-hat step arrived before the previous hit had finished, so the Clip had to be stopped and restarted, which cut the sound off and made playback feel laggy and uneven. This was one of two causes of the lag we noticed; the other is described under Topic 2. Giving each instrument several Clips in rotation fixed this, because a new hit no longer interrupts the previous one. The main lesson was that Java Sound objects have state and a lifetime that must be designed around: format compatibility, resource cleanup, and what happens when a sample is changed while the program is running. One limitation remains: `Clip.start()` has some start-up latency of its own that we cannot control from Java. Audio turned the grid from a visual pattern into a working instrument, and the choice of samples gives users different drum sounds from the same pattern.

Topic 2: Multithreading

Why we chose it and how it fits the project

The sequencer must advance through 16 steps at a tempo-dependent speed, trigger sounds and move the playhead, while the window stays responsive to the user. Running this loop on the Swing thread would freeze the interface, so a separate playback thread is a requirement of the project, not an optional extra. It also lets the user edit the pattern and change the tempo while the music is playing.

What we learned

We learned how threads run independently and how shared data creates race conditions. We learned that `volatile` makes simple flags visible between threads, that `synchronized` is needed to protect data that is

read and changed in several steps, and that Swing follows a single-thread rule: components may only be touched on the event-dispatch thread (EDT), with `SwingUtilities.invokeLater()` used to hand work to it.

How we applied it

A playback thread loops over the 16 steps, and the length of a step is derived from the tempo (60 seconds / (BPM x 4)). The `playing` flag and the tempo are `volatile`, so the Stop button and the slider take effect immediately, and Stop also interrupts the thread's sleep instead of waiting for the current step to finish. The pattern is protected by a lock, and the playback thread copies the whole column of the current step under a single lock acquisition, so it never reads a half-edited step. The audio Clips are protected by a second, separate lock (see Topic 1). The playhead is only updated through `invokeLater`, which keeps timing and audio work separate from interface work.

Decisions

We used two separate locks so that editing the grid never has to wait for audio work, and the other way round. New samples are loaded outside the lock and only swapped in while holding it, so slow file reading cannot block playback. The playback thread is a daemon thread so it can never keep the program alive after the window closes. If a step ever runs late, the schedule is reset to the current time instead of trying to catch up, which avoids a burst of rapid steps after a delay.

Challenges and what we learned

1. Timing with Thread.sleep. Our first loop slept for one step length after every step. The work done in each step plus the overshoot of the sleep were added on top of every step, so the errors accumulated, the tempo drifted and the beat felt laggy. We fixed this by keeping a target time for the next step based on `System.nanoTime()`, adding one step length each time and sleeping only for the time that remains, so an error in one step is compensated instead of piling up.

2. Overlapping Clips. Together with the first problem, the cut-off Clips described in Topic 1 caused the audible lag, so the two fixes had to be made together before the playback sounded right.

3. Swing from the playback thread. We repeatedly updated the grid buttons (their colours) directly from the playback thread. Swing components are not thread-safe, so this is only allowed on the EDT. We moved every playhead update into `invokeLater` and kept the playhead state private to the EDT. Reviewing the code for thread safety also showed us a less obvious problem: the Clips were shared between the playback thread and the interface (changing a sample, closing the window), which is why they are now guarded by a lock.

Limitations and overall reflection

The timing issues are not gone entirely. `Thread.sleep` depends on the operating system scheduler, and garbage collection and Clip start-up latency add some jitter, especially with rapid hi-hat patterns. They are fixed well enough for the application to be functional. A sample-accurate sequencer would instead mix the audio itself into one `SourceDataLine` buffer, and loading samples could be moved off the EDT with a `SwingWorker`; we have not implemented either. The two topics are closely linked: the Clips are the point where the audio thread and the interface meet, and the hardest bugs were found there. The biggest lesson was the difference between making a sequence play and making it play consistently: a separate thread does not guarantee accurate timing, shared data must be managed carefully, and interface updates must happen on the right thread.